Rapport Énergétique

Projet Génie Logiciel

Équipe g125
Alexandre Danhiez, Noah Deplace, Ilès Ouyahia,
Nicolas Montauriol, Nathan Vigouroux
Ensimag

Janvier 2026

Résumé

Ce document présente l'analyse de l'empreinte énergétique de notre compilateur `decac`, tant du point de vue du code généré que du processus de développement.

Table des matières

1 Introduction **2**

2 Moyens de mesure et méthodologie **2**

3 Efficience du code produit (Compilateur) **2**
3.1 Stratégie d'optimisation par niveaux 2

4 Efficience du procédé de fabrication (Projet) **4**
4.1 Parallélisation des scripts de validation 4

5 Extension RISC-V **5**
5.1 Le coût énergétique du décodage 5
5.2 Architecture Load/Store et maîtrise du bus 5
5.3 Pression sur les registres et "Spilling" 5

6 Propositions et Pistes d'amélioration **6**
6.1 Améliorations algorithmiques 6
6.2 Améliorations méthodologiques 6

7 Conclusion **6**

1 Introduction

Dans un contexte actuel marqué par le réchauffement climatique et la surconsommation, la nécessité de réduire l'empreinte carbone du numérique est devenue un enjeu primordial. Il est donc logique que le projet de développement du compilateur `decac` doive se soumettre à ce genre de contrainte.

L'objectif de ce document est d'analyser l'impact énergétique de notre projet selon deux dimensions complémentaires :

— **L'efficience du produit logiciel :** Il s'agit d'évaluer la consommation des programmes générés par notre compilateur. Un code optimisé réduit le nombre de cycles processeur et le temps d'exécution, diminuant directement la consommation à l'exécution.

— **L'efficience du procédé de fabrication :** Le développement logiciel lui-même consomme des ressources (compilation, intégration continue, tests). Nous analyserons comment l'optimisation de nos chaînes de validation permet de réduire cette consommation.

2 Moyens de mesure et méthodologie

Afin d'évaluer l'impact énergétique de notre projet, nous avons adopté une approche en deux parties, distinguant l'efficience du code assembleur généré par notre compilateur (`decac`) et l'efficience du processus de fabrication (compilation, tests et intégration continue). Nous avons alors retenu les deux mesures suivantes :

— **1. Le nombre d'instructions :** Pour évaluer l'efficience des programmes générés par notre compilateur `decac`, nous mesurons le nombre total d'instructions exécutées par la machine virtuelle `ima`.

  — *Justification :* Cette approche permet de quantifier précisément l'efficacité de nos algorithmes de traduction (gestion des registres, optimisation des sauts, coût des structures objets). Moins il y a d'instructions pour une tâche donnée, moins le processeur est sollicité. Le coût énergétique est donc amoindri.

— **2. Le temps d'exécution (Procédé de fabrication) :** Pour évaluer le coût énergétique du développement (compilation et validation), nous mesurons le temps d'exécution.

  — *Justification :* La consommation d'énergie est directement corrélée à la durée d'exécution. En réduisant le temps d'exécution de nos tests et de la compilation, nous réduisons certainement l'énergie totale consommée par les serveurs d'intégration continue ou nos machines personnelles.

3 Efficience du code produit (Compilateur)

Cette section analyse la performance énergétique des programmes générés par notre compilateur `decac`. Comme défini dans notre méthodologie, nous utilisons le **nombre d'instructions** exécutées par la machine `ima` et le **temps d'exécution** comme indicateurs énergétiques principaux.

3.1 Stratégie d'optimisation par niveaux

Afin de permettre un compromis entre temps de compilation, sécurité et performance à l'exécution, nous avons implémenté un système d'optimisation incrémental activé par l'option `-o` du compilateur decac.

Niveau 1 : Simplifications locales (`-o`)

Ce premier niveau se concentre sur le nettoyage du code assembleur généré sans en modifier la structure de contrôle. Il agit sur des fenêtres d'instructions restreintes.

- **Élimination des redondances :** Suppression des séquences `LOAD/STORE` superflues lorsque la valeur est déjà présente dans un registre ou calculée "en place".
- **Réordonnancement commutatif (Sethi-Ullman) :** Pour les opérations `+` et `*`, nous permutons les opérandes pour placer le terme le plus complexe à gauche, optimisant ainsi l'usage des registres.
- **Réécritures arithmétiques :** Certaines opérations coûteuses sont remplacées par des équivalents moins gourmands.
- *Exemple concret :* Dans le fichier `ln2.deca`, l'expression `(g+d) / 2.0` est réécrite en une multiplication par `0.5`, évitant l'usage de l'instruction `DIV` (coûteuse) sans ajouter de branchement. De même, les comparaisons `x == 0` utilisent des instructions immédiates optimisées.

Niveau 2 : Calculs anticipés (`-o -o`)

Ce niveau introduit l'évaluation partielle à la compilation (Constant Folding and Propagation).

- **Principe :** Dès qu'une sous-expression est composée d'opérandes constants, elle est évaluée par le compilateur et remplacée par sa valeur littérale (`ImmediateFloat` ou `ImmediateInteger`).
- *Exemple concret :* Dans `ln2.deca`, les divisions de la série de Taylor (ex : `1.0/5040.0`) sont toutes pré-calculées. Cela réduit significativement la taille du code binaire et la pression sur les registres à l'exécution.

Niveau 3 : Registres dédiés et Strength Reduction (`-o -o -o`)

Ce niveau active des transformations qui modifient l'allocation des ressources et la structure du graphe de flot de contrôle.

- **Promotion de variables :** Les variables locales identifiées comme critiques sont "épinglées" dans des registres sauvegardés, supprimant totalement les accès mémoire (pile) pour ces variables.
- **Strength Reduction (Divisions par 2^k) :** Les divisions et modulos entiers par une puissance de 2 sont remplacés par des décalages de bits.
- **Gestion du "Hot Path" :** Ces optimisations par décalage nécessitent une garde (un `if`) pour gérer correctement les nombres négatifs. Bien que cela ajoute potentiellement des instructions de branchement, le gain de performance sur le chemin principal (nombres positifs) est net.
- *Exemple concret :* Dans `Syracuse.deca`, les opérations `u/2` et `u%2` deviennent de simples décalages logiques.

Mode "Sans Filet" : Suppression des sécurités (`-n`)

L'option `-n` permet de mesurer la performance brute du code ("Raw Performance").

- **Action :** Elle désactive la génération systématique des tests de débordement (`BOV`), de division par zéro et de déréférencement nul.
- **Impact :** Le flux d'instructions est fluide, sans interruption par des instructions de comparaison et de branchement conditionnel après chaque calcul.

— **Usage :** Strictement réservé au benchmarking sur des algorithmes certifiés corrects, car le comportement devient indéfini en cas d'erreur.

Toutes ces optimisations permettent de réduire drastiquement le nombre d'instructions et le temps d'exécution des fichiers compilés par notre code. On peut observer par exemple que le nombre d'instructions des programmes *ln2.deca* et *ln2_fct.deca* a significativement chuté après l'implémentation de chaque option d'optimisation. Cela mène donc inévitablement à une baisse du coût énergétique du code fourni.

De la même manière, ces optimisations permettent également une grande réduction des temps d'exécution. Comme on peut le constater dans les trois exemples de la figure ci-dessous, le temps d'exécution est divisé par deux voire par trois à l'issue des optimisations. Ce qui entraîne également une réduction de la consommation énergétique du compilateur decac.



FIGURE 1 – Performances pour les fichiers ln2.deca, ln2_fct.deca et syracuse.deca

4 Efficience du procédé de fabrication (Projet)

La phase de validation (ensemble des scripts de test) est l'étape la plus coûteuse en ressources dans le cycle de vie du développement. Une exécution séquentielle de l'intégralité de la suite de tests à chaque modification serait énergivore et chronophage. Nous avons donc repensé notre chaîne de validation.

4.1 Parallélisation des scripts de validation

Le script de validation fourni initialement exécutait les tests de manière séquentielle. Sur une suite de nombreux tests, cela sous-utilise les capacités de nos machines tout en les maintenant actives plus longtemps.

Nous avons refondu nos scripts de validation (Python/Shell) pour exploiter le parallélisme de données :

— **Mécanisme :** Utilisation de l'option -P du compilateur pour paralléliser la compilation de nos fichiers de test.

— **Impact sur la pipeline Git :** Chaque ajout de code sur le Git déclenchant des tests de régression, cela permet donc une réduction de l'impact énergétique de la pipeline.

— **Impact énergétique :** Bien que la puissance instantanée augmente car la machine est plus sollicitée, la durée totale du traitement diminue drastiquement. L'énergie totale est réduite car la consommation de l'ordinateur est amortie sur une durée bien plus courte.

Ce gain de temps a un double bénéfice : il réduit la consommation électrique globale par cycle de validation et améliore l'expérience développeur.

5 Extension RISC-V

Le compilateur développé dans ce projet cible la machine virtuelle `ima`. Si ce choix est très cohérent pour la pédagogie, il est aujourd'hui loin d'être optimisé en termes d'efficacité énergétique.

Une migration vers une cible RISC-V apporte donc des gains structurels majeurs, basés sur les principes physiques suivants :

5.1 Le coût énergétique du décodage

L'énergie consommée par un processeur se divise en deux : l'énergie de calcul (utile) et l'énergie de contrôle (administrative).

— **Sur 68k :** Les instructions sont de taille variable et très complexes. Le processeur doit intégrer un décodeur lourd pour interpréter chaque instruction. Ce décodeur consomme une quantité significative d'énergie à chaque cycle, avant même que le moindre calcul ne soit effectué.

— **Sur RISC-V :** Le format d'instruction est fixe (32 bits) et régulier. Le décodage est "câblé" et nécessite très peu de portes logiques.

— **Bilan :** RISC-V élimine la "taxe de décodage", permettant de consacrer la quasi-totalité de l'énergie aux unités arithmétiques et logiques (ALU).

5.2 Architecture Load/Store et maîtrise du bus

L'accès à la mémoire est l'opération la plus coûteuse énergétiquement.

— **Sur 68k :** L'architecture autorise les opérandes mémoire directs (ex : `ADD (A0), D1`). Cela masque les accès bus et empêche le compilateur de voir le coût réel. De plus, les modes d'adressage complexes sollicitent l'unité de calcul d'adresse inutilement.

— **Sur RISC-V :** C'est une architecture *Load/Store* stricte. Seules les instructions `LW` et `SW` accèdent à la mémoire. Le compilateur a donc un contrôle total et explicite sur le trafic mémoire. Il peut optimiser l'ordonnancement pour grouper les accès ou les éviter, réduisant la consommation dynamique des bus de données.

5.3 Pression sur les registres et "Spilling"

L'énergie la moins chère est celle des données qui ne sortent jamais du processeur.

— **Sur 68k :** Les registres sont partitionnés en deux familles disjointes : 8 registres de Données (`D0-D7`) et 8 d'Adresses (`A0-A7`). Cette rigidité complique l'allocation de registres (coloration de graphe) et force souvent le compilateur à faire du "spilling" (sauvegarder temporairement un registre en pile pour le libérer), générant des écritures mémoire inutiles.

— **Sur RISC-V :** Avec 31 registres généraux unifiés (`x1-x31`), l'allocateur dispose d'une bien plus grande liberté pour maintenir les variables locales "en vie" dans le cœur du processeur, minimisant les interactions avec la pile (Stack).

Conclusion : Si le 68k privilégie la densité sémantique (faire beaucoup avec une instruction), le RISC-V privilégie l'efficience thermique. RISC-V est donc un choix très cohérent en vue d'une réduction de la consommation énergétique de notre compilateur.

6 Propositions et Pistes d'amélioration

Bien que notre compilateur `decac` intègre déjà plusieurs niveaux d'optimisation, la contrainte de temps inhérente au projet ne nous a pas permis d'explorer toutes les stratégies de réduction de l'empreinte énergétique. Voici les pistes que nous privilégierions dans une optique de développement futur d'optimisations énergétiques.

6.1 Améliorations algorithmiques

L'objectif est de réduire le nombre de cycles CPU et les accès mémoire, qui sont les opérations les plus coûteuses énergétiquement.

1. **Allocation de registres globale (Coloration de Graphe) :** Actuellement, notre gestion des registres est locale ou basée sur une pile. L'implémentation de l'algorithme de *Chaitin* (coloration de graphe d'interférence) permettrait de mapper un maximum de variables temporaires et locales directement dans les registres physiques.
   — *Gain espéré :* Une réduction drastique des instructions `LOAD` et `STORE` (coût mémoire), remplacées par des opérations registre-à-registre beaucoup plus sobres.
2. **Optimisation des boucles (Loop Invariant Code Motion) :** Les boucles étant les zones les plus exécutées ("Hot Spots"), toute optimisation y est critique. Nous proposons de détecter les calculs invariants (expressions qui ne changent pas d'une itération à l'autre) pour les sortir de la boucle.
   — *Gain espéré :* Si une boucle tourne 1000 fois, sortir une addition permet d'économiser 999 opérations inutiles.
3. **Élimination du Code Mort (Dead Code Elimination) :** L'analyse du graphe de flot de contrôle (CFG) permettrait de détecter des blocs de code inaccessibles (ex : après un `return` ou dans un `if (false)`).
   — *Gain espéré :* Réduction de la taille du binaire (moins de stockage/transfert) et suppression de sauts inutiles.

6.2 Améliorations méthodologiques

Au-delà du code, le processus de développement peut être rendu plus écologique.

1. **Profilage énergétique dans la CI/CD :** Intégrer un outil de mesure automatique (comme *PowerAPI* ou un compteur de cycles strict) directement dans la pipeline Git.
   — *Objectif :* Rejeter automatiquement un commit qui augmenterait trop la consommation énergétique (non-régression énergétique).
2. **Mutualisation des tests :** Plutôt que chaque groupe d'étudiants fasse tourner sa propre batterie de tests sur des machines séparées, utiliser un serveur de cache partagé pour les résultats de compilation des bibliothèques communes.

7 Conclusion

Ce projet de Génie Logiciel nous a permis de constater que l'efficience énergétique n'est pas une propriété magique, mais la conséquence directe de choix d'implémentation précis.

Nous avons démontré qu'en passant d'une compilation naïve à une compilation optimisée (Niveau 3), le nombre de cycles processeur peut être réduit significativement. Parallèlement, l'optimisation de notre chaîne de tests a permis de réduire le temps machine nécessaire à la validation.

Dans un contexte mondial de sobriété numérique, le rôle du développeur de compilateur est central : chaque optimisation implémentée ici est une économie d'énergie multipliée par le nombre de fois où les programmes utilisateurs seront exécutés.